

DATE:

Lab 4

Aim: Based on the requirements in OO design assignments, create inheritance hierarchy and implement abstract class, interfaces

Code:

// Interface

```
interface Printable {  
    void printDetails();  
}
```

// Abstract Class

```
abstract class Employee implements Printable {  
    protected int id;  
    protected String name;
```

// Constructor

```
public Employee (int id, String name) {  
    this.id = id;  
    this.name = name;  
}
```

```
public abstract double calculateSalary();  
}
```

```
class FullTimeEmployee extends Employee {  
    private double monthlySalary;  
    public FullTimeEmployee (int id, String name,  
        double monthlySalary) {
```



DATE:

```
super(id, name);  
this.monthlySalary = monthlySalary;  
}
```

@Override

```
public double calculateSalary() {  
    return monthlySalary;  
}
```

@Override

```
public void printDetails() {  
    System.out.println("Full-Time Employee");  
    System.out.println("ID: " + id);  
    System.out.println("Name: " + name);  
    System.out.println("Salary: ₹" + calculateSalary());  
    System.out.println();  
}
```

}

```
class PartTimeEmployee extends Employee {  
    private int hoursWorked;  
    private double hourlyRate;
```

```
    public PartTimeEmployee(int id, String name, int  
        hoursWorked, double hourlyRate) {  
        super(id, name);  
        this.hoursWorked = hoursWorked;  
        this.hourlyRate = hourlyRate;  
    }
```

@Override

```
public double calculateSalary() {  
    return hoursWorked * hourlyRate;  
}
```

Output:

Full Time Employee

ID : 101

Name : Rahul

Salary : ₹ 50000.0

Part Time Employee

ID : 102

Name : Priya

Salary : ₹ 40000.0





@Override

```
public void printDetails() {
```

```
    System.out.println("Part-Time Employee");
```

```
    System.out.println("ID: " + id);
```

```
    System.out.println("Name: " + name);
```

```
    System.out.println("Salary: ₹ " + calculateSalary());
```

```
    System.out.println();
```

```
}
}
```

```
public class EmployeeManagement {
```

```
    public static void main (String[] args) {
```

```
        Employee emp1 = new FullTimeEmployee
```

```
            (101, "Rohit", 50000);
```

```
        Employee emp2 = new PartTimeEmployee
```

```
            (102, "Priya", 30000);
```

```
        emp1.printDetails();
```

```
        emp2.printDetails();
```

```
    }
```

```
}
```



Program 2:

```
interface PaymentGateway {
    void processPayment (double amount);
}
```

```
abstract class Payment implements PaymentGateway {
    protected String transactionId;
    protected String customerName;
```

```
public class Payment implements PaymentGateway {
```

```
    public Payment(String transactionId, String
        customerName) {
        this.transactionId = transactionId;
        this.customerName = customerName;
    }
```

```
    public abstract void showReceipt (double
        amount);
}
```

```
class CreditCardPayment extends Payment {
    private String cardNumber;
    public CreditCardPayment (String transactionId,
        String customerName, String cardNumber) {
        System.out.println ("Processing credit card
        payment of ₹" + amount + " for card ending in
        " + cardNumber);
    }
```



@Override

```
public void processPayment (double amount){
    System.out.println("Processing Credit Card  

    payment of ₹" + amount + " for card ending  

    + cardNumber.substring(cardNumber.length()-4);
    showReceipt (amount);
}
```

@Override

```
public void showReceipt (double amount){
    System.out.println("Mode: Credit Card");
    System.out.println("Transaction ID: " + transaction  

    Id);
    System.out.println("Customer: " + customer  

    Name);
    System.out.println("Amount Paid: ₹" +  

    amount);
    System.out.println();
}
```

```
class UPIPayment extends Payment {
    private String upiId;
```

```
public UPIPayment (String transactionId, String  

    customerName, String upiId){
    super (transactionId, customerName);
    this.upiId = upiId;
}
```

@Override

```
public void processPayment (double amount){
    System.out.println("Processing UPI payment of ₹"  

    + amount + " for VPA: " + upiId);
}
```

Output

Processing Credit Card payment of ₹2500.0 for card ending 1234 56

Mode: Credit Card.

Transaction ID : TXN1001

Customer : Aman Verma

Amount Paid : ₹2500.0

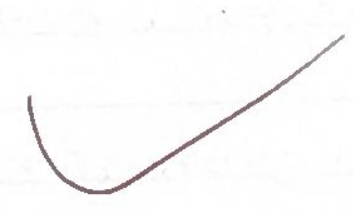
Processing UPI payment of ₹750.5 for NPA : snoko@00

Mode : UPI

Transaction ID : TXN1002

Customer : snokoRao

Amount Paid : ₹750.5





DATE:

~~@ Override~~

```
public void processPayment (double amount) {  
    System.out.println ("Processing ");  
}
```

@ Override

```
public void showReceipt (double amount) {  
    System.out.println ("Mode: UPI");  
    System.out.println ("Transaction ID: " +  
        transactionId);  
    System.out.println ("customer: " + customerName);  
    System.out.println ("Amount Paid: ₹" + amount);  
}
```

```
public class PaymentManagement {  
    public class PaymentManagement {  
        public static void main (String[] args) {  
            PaymentGateway payment1 = new CreditCard  
                Payment ("TxN1001", "Aman Verma",  
                    "12345678910325");  
            PaymentGateway payment2 = new UPIPayment  
                ("TxN1002", "Sneha Rao", "sneha@okaxis");  
  
            payment1.processPayment (2500.0);  
            payment2.processPayment (750.50);  
        }  
    }  
}
```



Theory:

**Inheritance:** Inheritance is a fundamental concept of OOP that allows a derived class to acquire properties and behaviors from a base class.

**Abstract Class:** An abstract class declared with the abstract keyword that cannot be directly instantiated. It serves as a blueprint for other classes.

**Interface:** An Interface in Java is a reference type that specifies a contract of what class should do, without defining how.

**Conclusion:** Through this practical the principle of abstraction and polymorphism were successfully implemented using Java abstract and interfaces.



## Theory:

1] Inheritance : Inheritance is a fundamental concept of Object-Oriented Programming (OOP) that allows a derived class to acquire properties and behaviours from a base class

2] Abstract Class : An abstract class is a

Helix  
29/8/2020